Informe de laboratorio 04

Tema: Programación para el microprocesador INTEL - Instrucciones Lógicas y Registro de Banderas

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Arq. de computadoras Semestre: IV Código: 1702227

Laboratorio	Tema	Duración
04	Programación para el microprocesador INTEL - Instrucciones Lógicas y Registro de Banderas	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	13/10/2026	–/10/2026

Índice

1. Objetivos **3**

2. Materiales y equipos **3**

3. Actividades **3**
3.1. Preparación del entorno . . . 3
3.2. Primer programa: AND, OR y XOR con máscaras . . . 3
3.3. Segundo programa: XOR, NOT y control de banderas . . . 6
3.3.1. Bloques originales . . . 6
3.3.2. Variante con `xor ah, dl` . . . 7
3.3.3. Control directo de banderas . . . 9
3.4. Tercer programa: máscaras con AND y TEST . . . 10
3.4.1. Máscara 40h (bit D6) . . . 11
3.4.2. Máscara 08h (bit D3) . . . 12

4. Análisis de resultados **13**

5. Tarea complementaria **14**
5.1. Almacenamiento de los datos declarados . . . 14
5.2. Directivas DB y DW . . . 14
5.3. Programa de manipulación de bits . . . 14

6. Conclusiones **16**

1. Objetivos

Objetivo general. Analizar la forma en que se ejecutan algunas instrucciones lógicas observando cómo estas afectan al registro de estados.

Objetivos específicos.

- Examinar detenidamente la manera en que se ejecutan algunas instrucciones lógicas.
- Analizar la manera en que se modifica el registro de banderas, dependiendo del tipo de operación y los datos que operan.
- Identificar las características propias de las máscaras que se utilizan junto con las instrucciones lógicas.

2. Materiales y equipos

- Guía de laboratorio.
- Computadora PC con el programa EMU8086.

3. Actividades

3.1. Preparación del entorno

- Se ejecutó el emulador EMU8086 y se creó un nuevo documento tipo COM.
- Cada programa se escribió con su encabezado `#make_COM#` y `org 100h`, necesario para generar un archivo `.COM`.
- Para observar las banderas se usó el botón **flags** de la ventana del emulador junto con la ejecución paso a paso (*single step*).

Las banderas que se monitorearon se resumen en la Tabla 1.

Cuadro 1: Banderas del registro de estados utilizadas en la práctica.

Bandera	**Nombre**	**Se activa (=1) cuando...**
Z	Cero	el resultado de la operación es 0.
S	Signo	el bit más significativo del resultado es 1 (bit 7 en operaciones de 8 bits, bit 15 en las de 16 bits).
P	Paridad	el byte bajo del resultado tiene un número par de unos.
C, O	Acarreo y desbordamiento	`AND`, `OR` y `XOR` las dejan siempre en 0.
D, I	Dirección e interrupción	se modifican con instrucciones específicas (`STD`/`CLD`, `STI`/`CLI`).

3.2. Primer programa: AND, OR y XOR con máscaras

Objetivo. Observar cómo las instrucciones `AND`, `OR` y `XOR` operan sobre registros de 8 y 16 bits usando máscaras, y cómo cada una actualiza las banderas. El programa declara sus datos con las directivas `DB` y `DW`.

Funcionamiento.

- `DL = dato1 AND mascara1` (8 bits).
- `CX = dato2 AND mascara2` (16 bits).

- `CX = CX AND CX`: no cambia CX, solo actualiza las banderas.

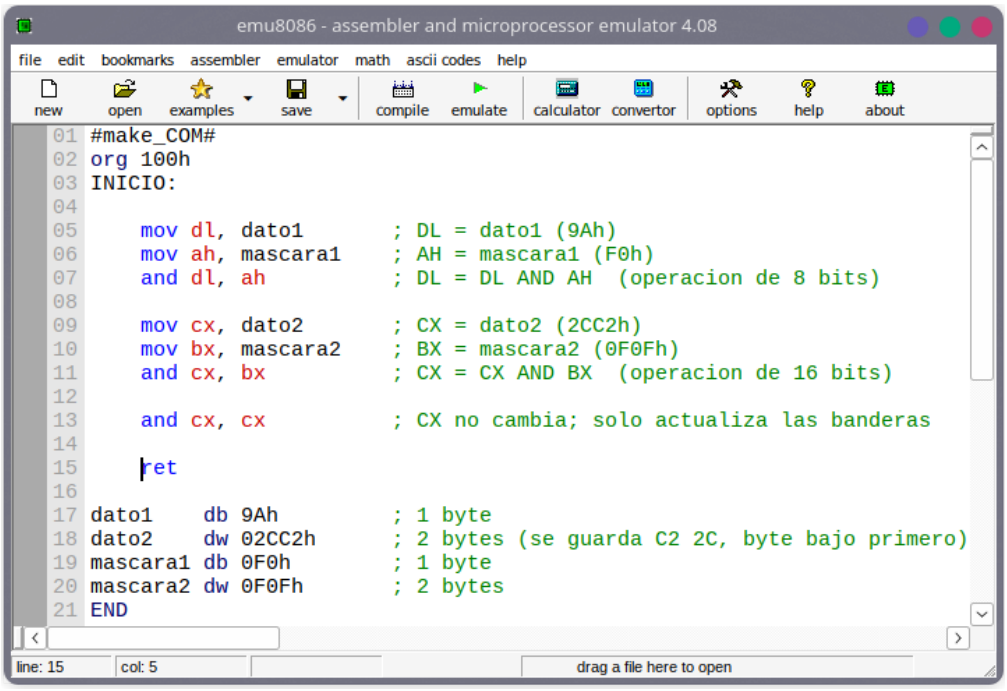


Figura 1: Código del primer programa en el editor de EMU8086.

Procedimiento.

- Se compiló y se ejecutó paso a paso, anotando tras cada `AND` los operandos, el resultado y las banderas (Tabla 2).
- Se sustituyeron las tres instrucciones `AND` por `OR` y se repitió el procedimiento (Tabla 3).
- Se sustituyeron por `XOR` y se repitió nuevamente (Tabla 4).

Cuadro 2: Resultados del primer programa con `AND`.

Instrucción	Operandos	Resultado	S	Z	P	C/O
`and dl, ah`	9Ah, F0h	90h	1	0	1	0/0
`and cx, bx`	2CC2h, 0F0Fh	0C02h	0	0	0	0/0
`and cx, cx`	0C02h, 0C02h	0C02h	0	0	0	0/0

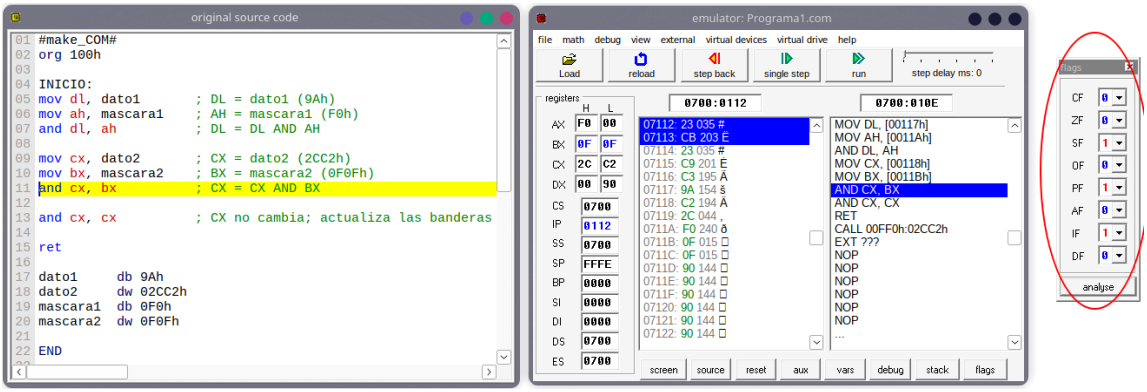


Figura 2: Registros y banderas ejecutando las instrucciones AND.

Cuadro 3: Resultados del primer programa con `OR`.

Instrucción	Operandos	Resultado	S	Z	P	C/O
`or dl, ah`	9Ah, F0h	FAh	1	0	1	0/0
`or cx, bx`	2CC2h, 0F0Fh	2FCFh	0	0	1	0/0
`or cx, cx`	2FCFh, 2FCFh	2FCFh	0	0	1	0/0

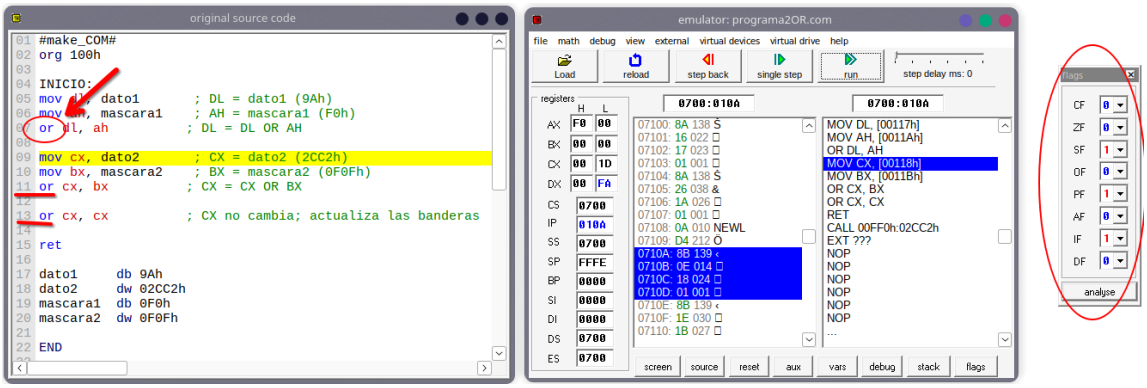


Figura 3: Registros y banderas ejecutando las instrucciones OR.

Cuadro 4: Resultados del primer programa con `XOR`.

Instrucción	Operandos	Resultado	S	Z	P	C/O
`xor dl, ah`	9Ah, F0h	6Ah	0	0	1	0/0
`xor cx, bx`	2CC2h, 0F0Fh	23CDh	0	0	0	0/0
`xor cx, cx`	23CDh, 23CDh	0000h	0	1	1	0/0

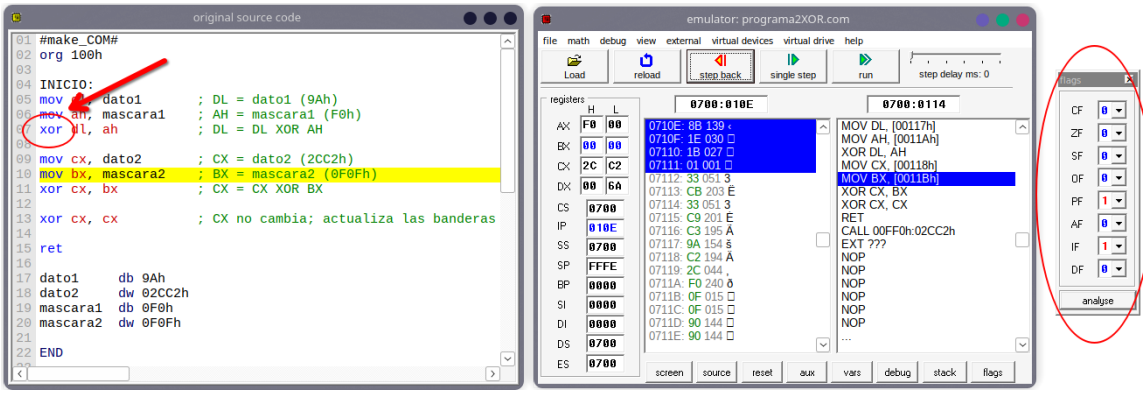


Figura 4: Registros y banderas ejecutando las instrucciones OR.

Observaciones. Con la máscara F0h, `AND` conserva el nibble alto y borra el bajo (90h), `OR` pone en 1 el nibble alto y conserva el bajo (FAh) y `XOR` invierte el nibble alto y conserva el bajo (6Ah). En las tres operaciones C y O quedaron en 0. Con `XOR` la tercera instrucción dejó CX en 0000h y activó Z.

3.3. Segundo programa: XOR, NOT y control de banderas

Objetivo. Comparar dos formas de invertir los bits de un dato (`XOR` con FFh y `NOT`), observar que obtienen el mismo resultado pero dejan banderas distintas, y comprobar que algunas banderas se pueden activar o desactivar con instrucciones específicas.

3.3.1. Bloques originales

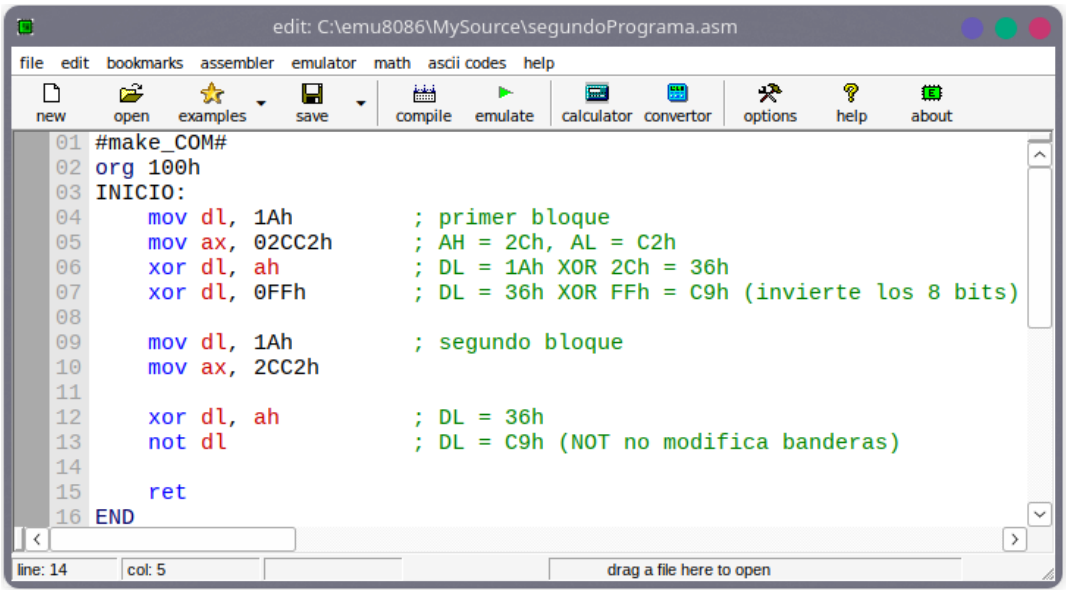


Figura 5: Código del segundo programa en el editor de EMU8086.

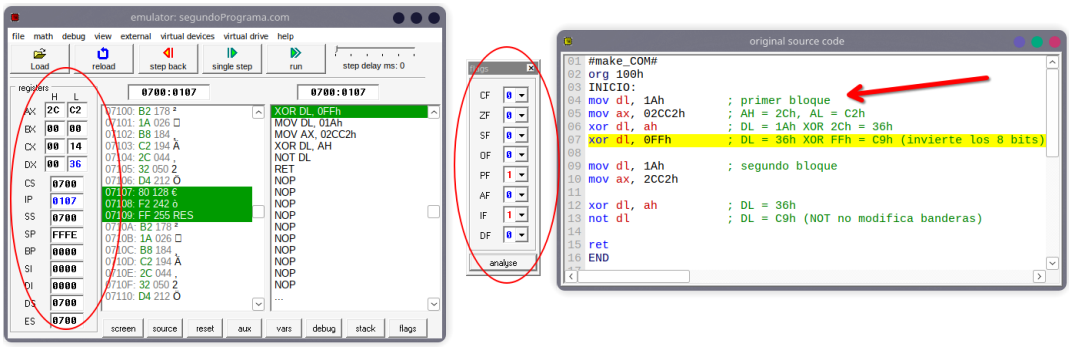


Figura 6: Registros y estado de flags en la ejecución del primer bloque.

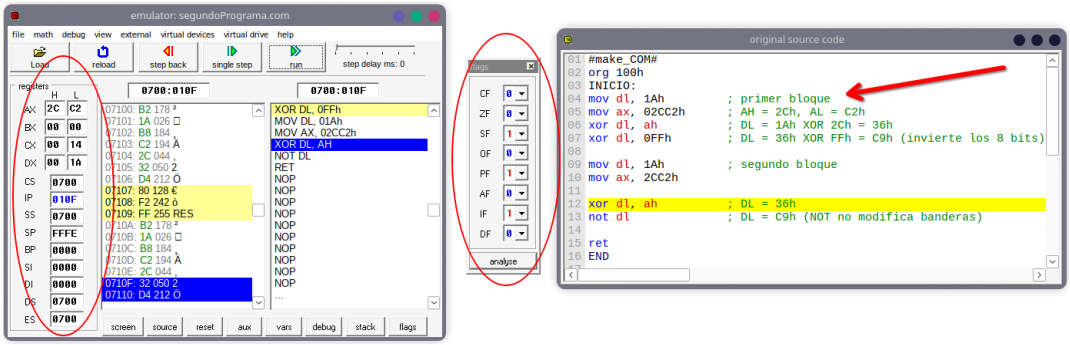


Figura 7: Registros y estado de flags en la ejecución del segundo bloque.

Cuadro 5: Ejecución paso a paso del segundo programa original.

Instrucción	Operación	DL	AH	S	Z	P
Primer bloque						
`mov dl, 1Ah`	carga	1Ah	–	–	–	–
`mov ax, 02CC2h`	carga	1Ah	2Ch	–	–	–
`xor dl, ah`	1Ah XOR 2Ch	36h	2Ch	0	0	1
`xor dl, 0FFh`	36h XOR FFh	C9h	2Ch	1	0	1
Segundo bloque						
`mov dl, 1Ah`	carga	1Ah	2Ch	–	–	–
`mov ax, 2CC2h`	carga	1Ah	2Ch	–	–	–
`xor dl, ah`	1Ah XOR 2Ch	36h	2Ch	0	0	1
`not dl`	NOT 36h	C9h	2Ch	0	0	1

(–) La instrucción no modifica esa bandera. `NOT` tampoco modifica ninguna bandera.

Observaciones. En ambos bloques DL termina en C9h, pero la bandera S queda en 1 en el primero y en 0 en el segundo: `XOR dl, 0FFh` es una operación lógica y actualiza las banderas con su resultado, mientras que `NOT dl` solo invierte los bits y conserva las banderas de la instrucción anterior.

3.3.2. Variante con `xor ah, dl`

Se modificó la tercera y la séptima instrucción del programa, sustituyendo `xor dl, ah` por `xor ah, dl`:

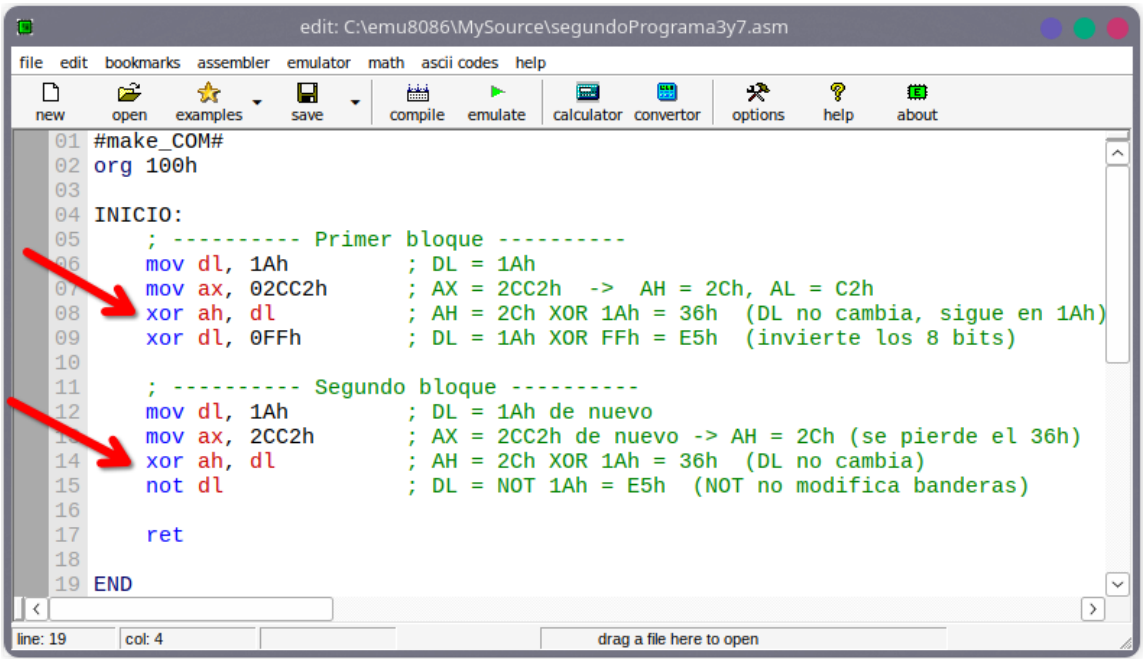


Figura 8: Bloques del segundo programa con `xor ah, dl`.

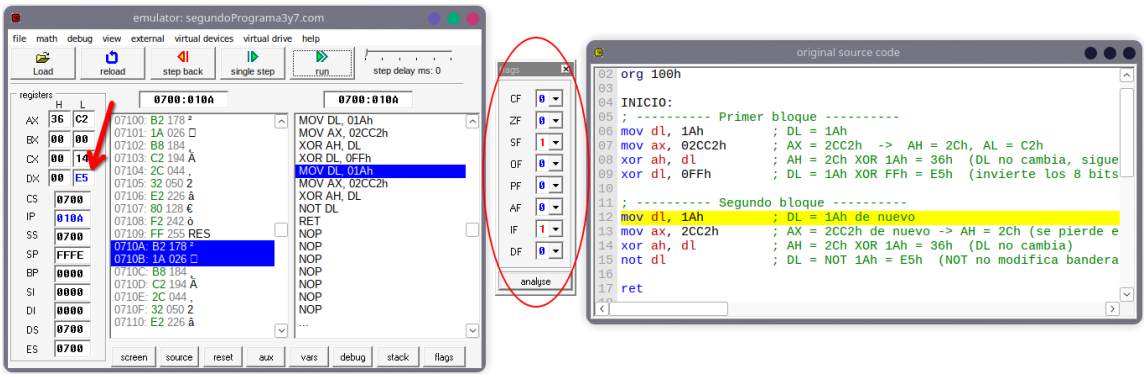


Figura 9: Estado de DL y de las banderas al final del primer bloque.

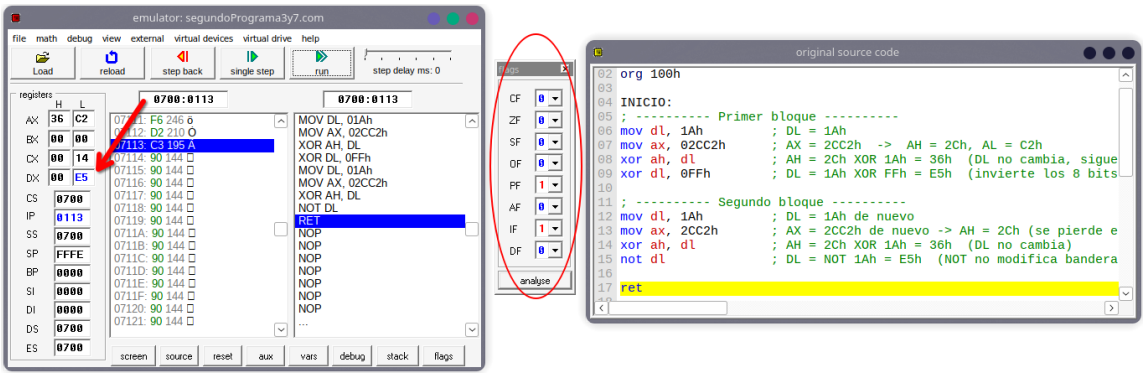


Figura 10: Estado de DL y de las banderas al final del segundo bloque

Estado de DL y de las banderas al final del segundo bloque

Cuadro 6: Ejecución del segundo programa con `xor ah, dl`.

Instrucción	Operación	DL	AH	S	Z	P
Primer bloque						
`xor ah, dl`	2Ch XOR 1Ah	1Ah	36h	0	0	1
`xor dl, 0FFh`	1Ah XOR FFh	E5h	36h	1	0	0
Segundo bloque						
`xor ah, dl`	2Ch XOR 1Ah	1Ah	36h	0	0	1
`not dl`	NOT 1Ah	E5h	36h	0	0	1

Observaciones. Ahora el resultado de la primera operación se guarda en AH (36h) y DL conserva su valor inicial (1Ah). Al final DL vale E5h en lugar de C9h, AH vale 36h en lugar de 2Ch, y en el primer bloque la bandera P pasa de 1 a 0 (E5h tiene cinco unos).

3.3.3. Control directo de banderas

Se insertaron instrucciones que modifican banderas sin depender del resultado de ninguna operación (se tomó como base el programa original):

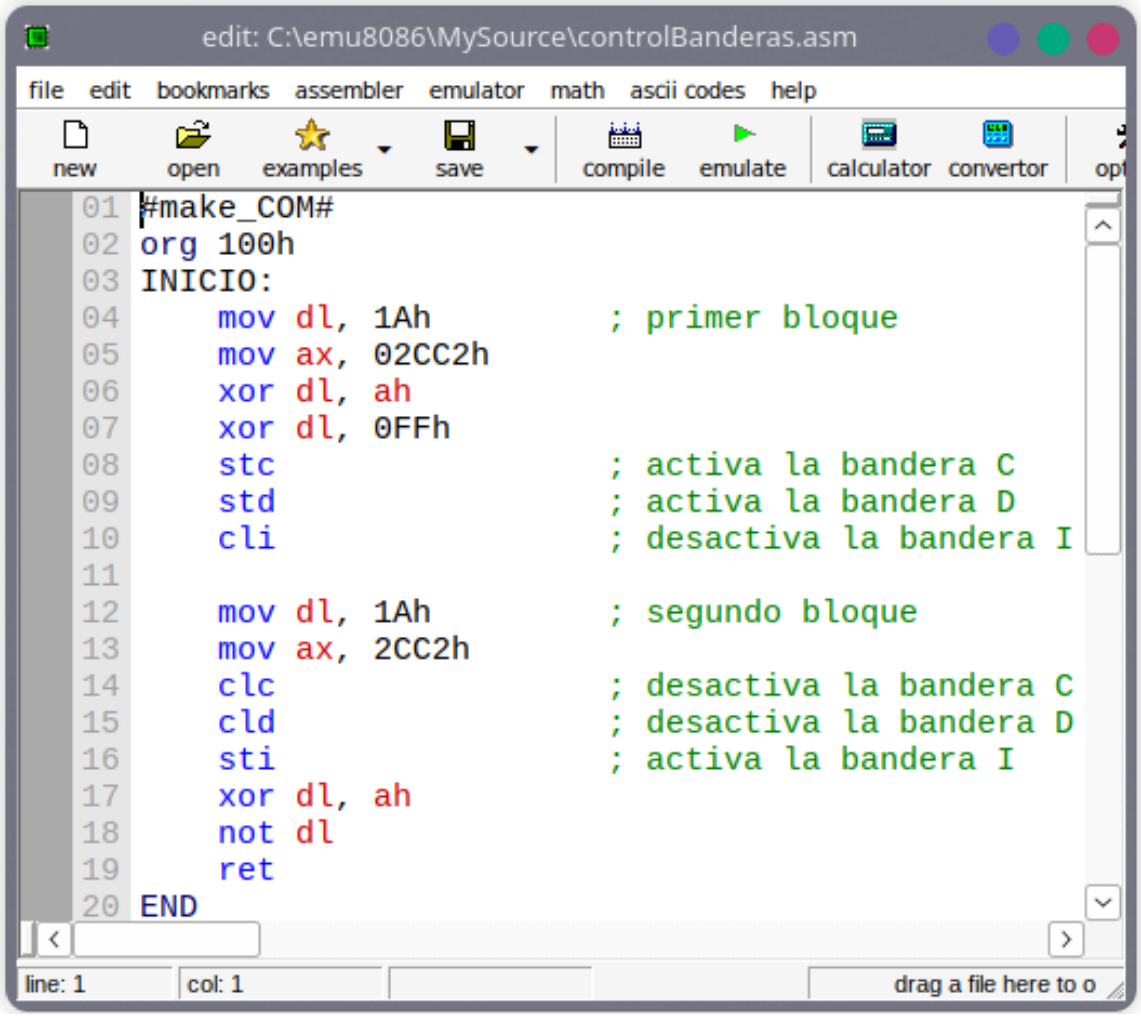


Figura 11: Bloques del segundo programa con instrucciones de control de banderas.

Cuadro 7: Efecto de las instrucciones de control de banderas.

Instrucción	Bandera	Valor	Observación
`stc`	C	1	Activa C sin importar ningún resultado.
`std`	D	1	Activa la bandera de dirección.
`cli`	I	0	Desactiva las interrupciones.
`clc`	C	0	Desactiva C.
`cld`	D	0	Desactiva la bandera de dirección.
`sti`	I	1	Activa las interrupciones.
`xor dl, ah`	S, Z, P, C, O	S=0, Z=0, P=1, C=O=0	Se actualizan según el resultado; D e I no cambian.

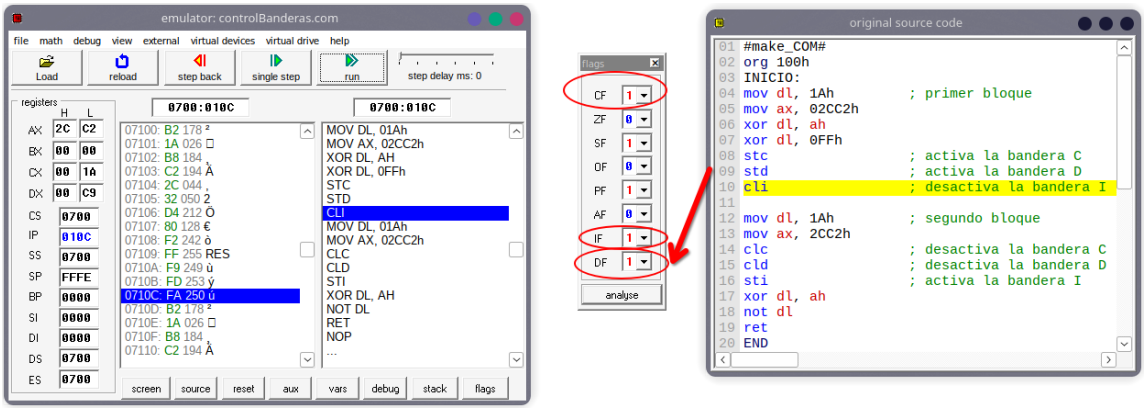


Figura 12: Emulando las instrucciones, banderas *C, D* e *I* modificadas.

Conclusión de la actividad. Existen dos tipos de banderas: las que dependen del resultado de una operación (S, Z, P, y también C y O en las operaciones lógicas) y las que se controlan explícitamente con instrucciones propias (C, D e I). Una operación lógica posterior puede cambiar C (la deja en 0), pero no altera D ni I.

3.4. Tercer programa: máscaras con AND y TEST

Objetivo. Aprender a comprobar el estado de un bit usando una máscara y comparar `AND` con `TEST`: ambas actualizan las banderas igual, pero `TEST` no guarda el resultado.

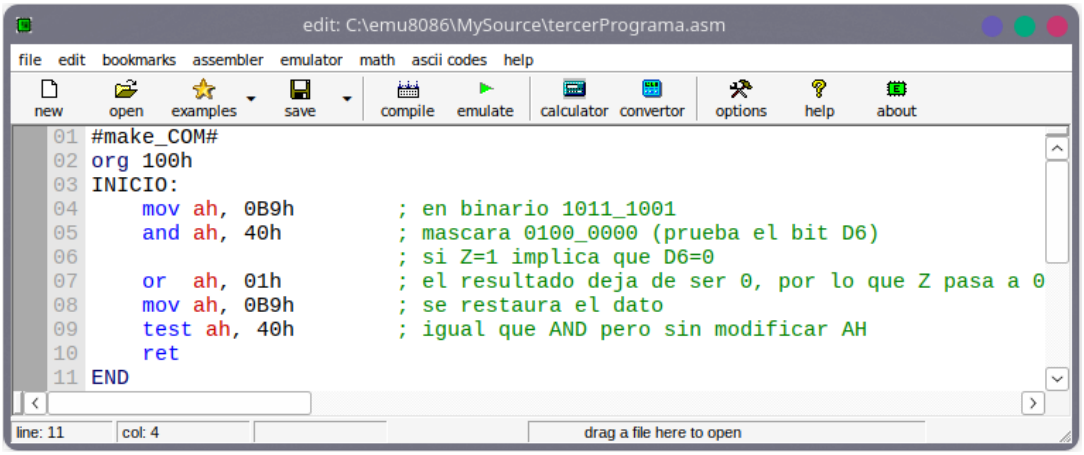


Figura 13: Código del tercer programa en el editor de EMU8086.

3.4.1. Máscara 40h (bit D6)

Cuadro 8: Ejecución del tercer programa con la máscara 40h.

Instrucción	AH	S	Z	P	Interpretación
`mov ah, 0B9h`	B9h	–	–	–	`MOV` no modifica banderas.
`and ah, 40h`	00h	0	1	1	Z=1: el bit D6 de B9h vale 0.
`or ah, 01h`	01h	0	0	0	Resultado distinto de cero.
`mov ah, 0B9h`	B9h	0	0	0	Se restaura AH; banderas sin cambio.
`test ah, 40h`	B9h	0	1	1	AH se conserva; Z=1 indica D6=0.

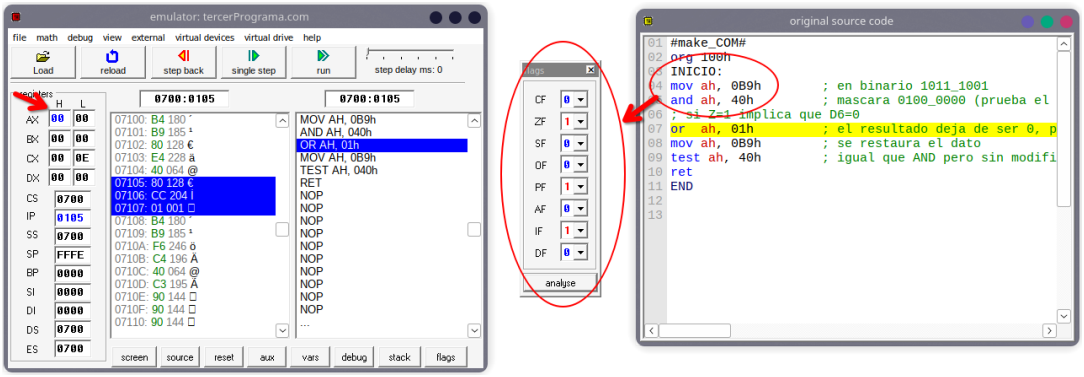


Figura 14: Emulador después de la instrucción *AND*.

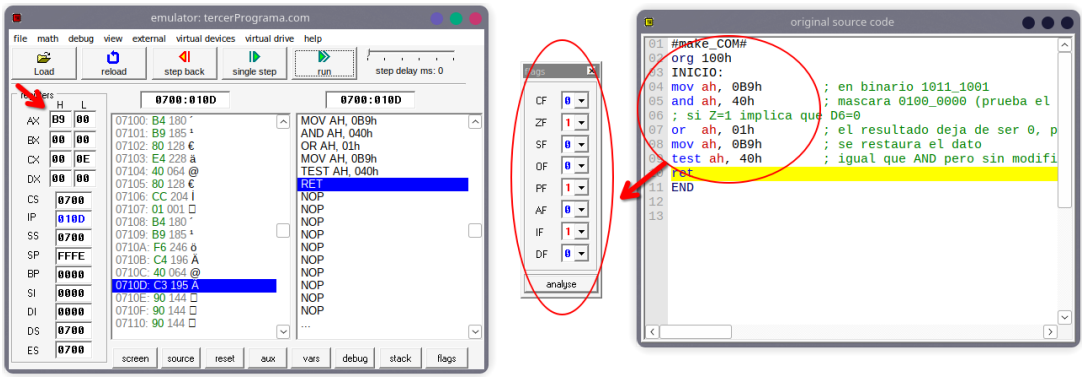


Figura 15: Emulador después de la instrucción *TEST*.

Interpretación. Al ejecutar las dos primeras instrucciones se activaron Z y P, y AH quedó en 00h. El resultado indica que el bit D6 del dato B9h (1011 1001) vale 0. Con `TEST` las banderas son las mismas, pero AH conserva su valor B9h.

3.4.2. Máscara 08h (bit D3)

Se cambió el dato 40h por 08h en la segunda y quinta instrucción.

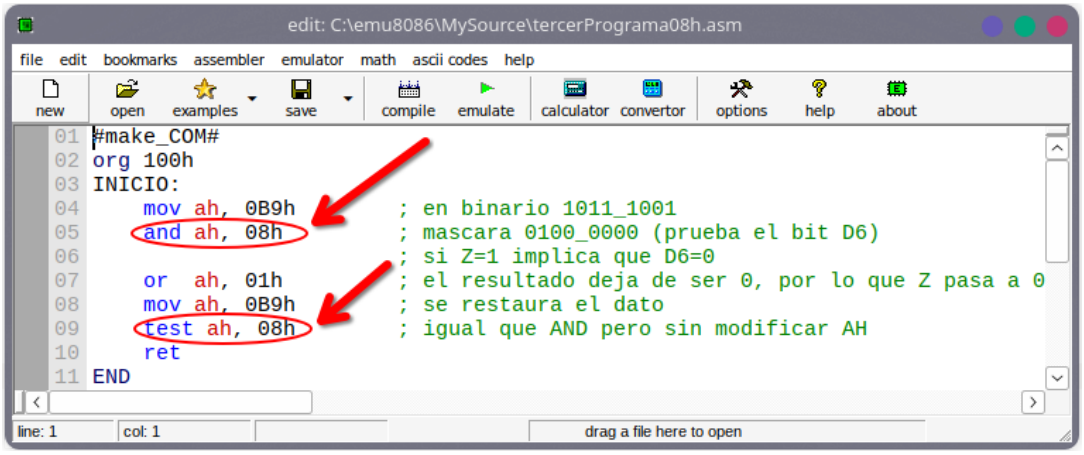


Figura 16: Emulador con la máscara 08h.

Cuadro 9: Ejecución del tercer programa con la máscara 08h.

Instrucción	AH	S	Z	P	Interpretación
`mov ah, 0B9h`	B9h	–	–	–	`MOV` no modifica banderas.
`and ah, 08h`	08h	0	0	0	Z=0: el bit D3 de B9h vale 1.
`or ah, 01h`	09h	0	0	1	Resultado distinto de cero.
`mov ah, 0B9h`	B9h	0	0	1	Se restaura AH; banderas sin cambio.
`test ah, 08h`	B9h	0	0	0	AH se conserva; Z=0 indica D3=1.

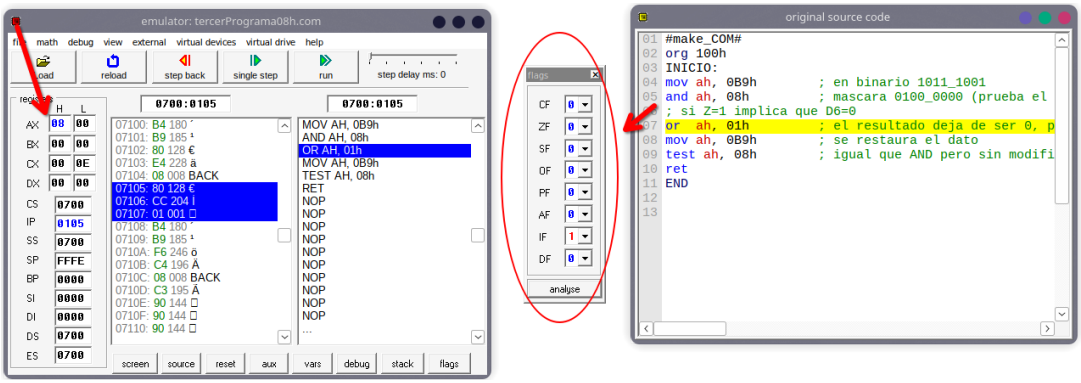


Figura 17: Emulador con la máscara *08h* después de la instrucción *AND*.

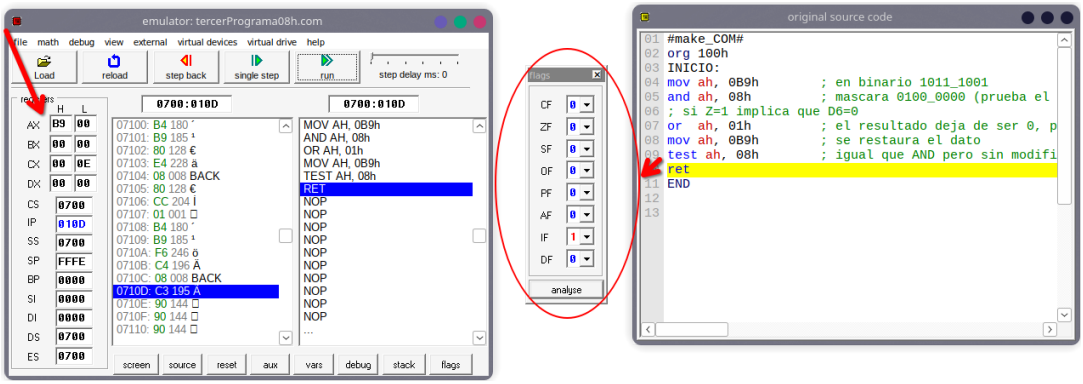


Figura 18: Emulador con la máscara *08h* después de la instrucción *TEST*.

Interpretación. Con la máscara 08h se prueba el bit D3. Como en B9h ese bit vale 1, el resultado de la operación es distinto de cero y la bandera Z queda en 0: el resultado del programa cambió respecto al caso de la máscara 40h.

4. Análisis de resultados

a) **Primer programa.**

1. **¿Qué objetivos se buscan al usar la máscara1 (F0h) con las tres diferentes instrucciones lógicas?**
   Actuar solo sobre el nibble alto (bits D7 a D4) y dejar intacto el bajo. Con `AND` se conserva el nibble alto y se borra el bajo (9Ah → 90h). Con `OR` se ponen en 1 los bits del nibble alto (9Ah → FAh). Con `XOR` se invierten los bits del nibble alto (9Ah → 6Ah). Así, la máscara permite borrar, poner en 1 o invertir un grupo de bits sin afectar a los demás.
2. **En la primera instrucción AND el bit D7 termina en alto y se activa la bandera S; en la segunda también termina en alto, pero no se activa la bandera, ¿por qué?**
   Porque la bandera S copia el bit más significativo *del tamaño de la operación*: el bit 7 en las operaciones de 8 bits y el bit 15 en las de 16 bits. La primera instrucción opera con registros de 8 bits (DL) y su bit 7 vale 1, por lo que S se activa. La segunda opera con registros de 16 bits (CX) y su bit 15 vale 0, por lo que S no se activa, aunque el bit 7 del resultado pueda valer 1 (por ejemplo, con `OR` el resultado 2FCFh tiene D7 = 1 y D15 = 0).
3. **Cuando se ejecuta la tercera instrucción lógica, con AND u OR las banderas reaccionan de forma similar, pero con XOR no, ¿por qué?**

Porque `and cx, cx` y `or cx, cx` devuelven el mismo valor de CX (A AND A = A y A OR A = A), de modo que las banderas quedan igual que tras la operación anterior. En cambio `xor cx, cx` da siempre 0 (A XOR A = 0): CX se borra y se activa Z.

4. **En el programa se declaran datos para hacer las operaciones, ¿dónde se guardan estos datos?**
   En la memoria, dentro del mismo segmento del programa (en un archivo `.COM`, CS = DS), a continuación de la última instrucción. Las instrucciones `mov` los leen con direccionamiento directo.

b) **Segundo programa.**

1. **El resultado del primer y del segundo bloque es idéntico en DL, pero el estado de las banderas no lo es, ¿por qué?**
   Porque `xor dl, 0FFh` es una operación lógica que actualiza las banderas según su resultado (C9h: S=1, Z=0, P=1), mientras que `not dl` solo invierte los bits y no modifica ninguna bandera. En el segundo bloque las banderas quedan como las dejó `xor dl, ah` (resultado 36h: S=0, Z=0, P=1).
2. **Al modificar el programa, tanto los resultados como las banderas terminaron en distintos estados, ¿cuáles fueron los cambios específicos que observó?**
   Con `xor ah, dl` el resultado se guarda en AH (36h) y DL conserva 1Ah. Después, `xor dl, 0FFh` y `not dl` dejan DL en E5h en lugar de C9h. En el primer bloque la bandera P pasó de 1 a 0 (E5h tiene cinco unos) y S sigue en 1. En el segundo bloque las banderas quedan en S=0, Z=0, P=1.

c) **Tercer programa.**

1. **¿Cuál es la diferencia fundamental entre las instrucciones AND y TEST?**
   Ambas realizan la operación AND bit a bit y actualizan las banderas de la misma forma, pero `AND` guarda el resultado en el operando destino (modificándolo), mientras que `TEST` descarta el resultado y deja intacto el dato. Por eso, tras `test ah, 40h`, AH conserva el valor B9h.
2. **¿Qué relación existe entre los bits que deseamos probar y el valor específico de la máscara seleccionada?**
   La máscara debe tener 1 en las posiciones de los bits que se quieren probar y 0 en las demás. Si tras el `AND` o `TEST` Z=1, todos los bits probados valen 0; si Z=0, al menos uno vale 1. Por ejemplo, 40h (0100 0000) prueba el bit D6 y 08h (0000 1000) prueba el bit D3.

5. Tarea complementaria

5.1. Almacenamiento de los datos declarados

Los datos que declara el programa se almacenan en la memoria, dentro del mismo segmento donde está el código (en un `.COM`, CS = DS), a continuación de la última instrucción (después de `ret`). El ensamblador asigna a cada etiqueta (`dato1`, `dato2`, ...) el desplazamiento que le corresponde.

5.2. Directivas DB y DW

- `DB` (*Define Byte*): reserva 1 byte (8 bits) por cada valor y lo inicializa. Ejemplo: `dato1 db 9Ah`.
- `DW` (*Define Word*): reserva 2 bytes (16 bits) por cada valor. En memoria se guarda primero el byte menos significativo (*little endian*): `dato2 dw 02CC2h` queda como C2h seguido de 2Ch.
- No son instrucciones del procesador sino directivas del ensamblador: no se ejecutan, solo indican cómo organizar los datos en memoria.

5.3. Programa de manipulación de bits

Enunciado. Dado un dato almacenado en AX, invertir los bits impares desde D5 hasta D1, borrar los bits pares desde D6 hasta D2 y poner en 1 los bits D7 y D0.

Solución. Se trabaja sobre AL (bits D7 a D0) con tres máscaras:

- `XOR` con 0010 1010b (2Ah): invierte D5, D3 y D1.
- `AND` con 1010 1011b (ABh): borra D6, D4 y D2 y conserva los demás.
- `OR` con 1000 0001b (81h): pone en 1 D7 y D0.

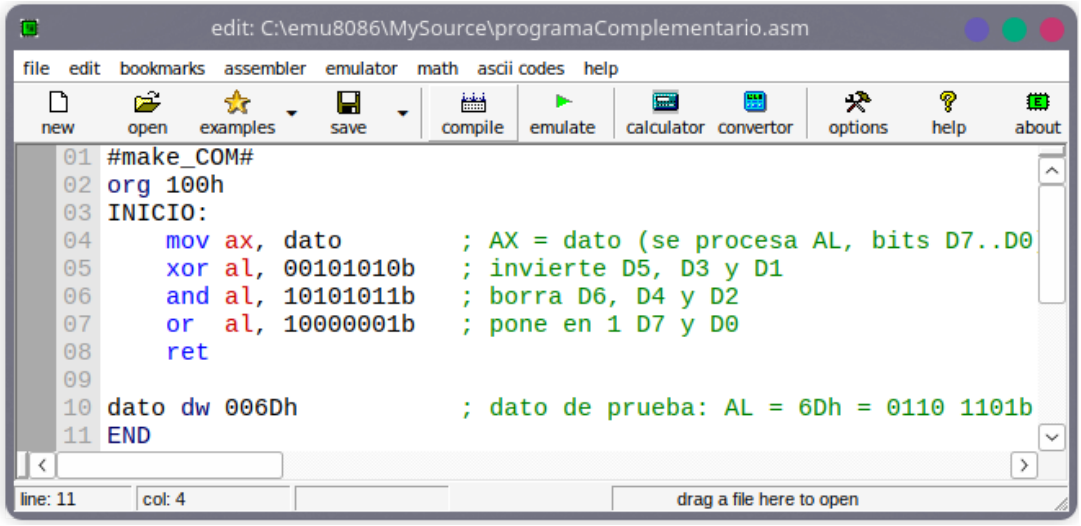


Figura 19: Código del programa de manipulación de bits.

Cuadro 10: Seguimiento del programa con el dato de prueba 6Dh.

Instrucción	**AL (hex)**	**AL (binario)**
`mov ax, dato`	6Dh	0110 1101
`xor al, 00101010b`	47h	0100 0111
`and al, 10101011b`	03h	0000 0011
`or al, 10000001b`	83h	1000 0011

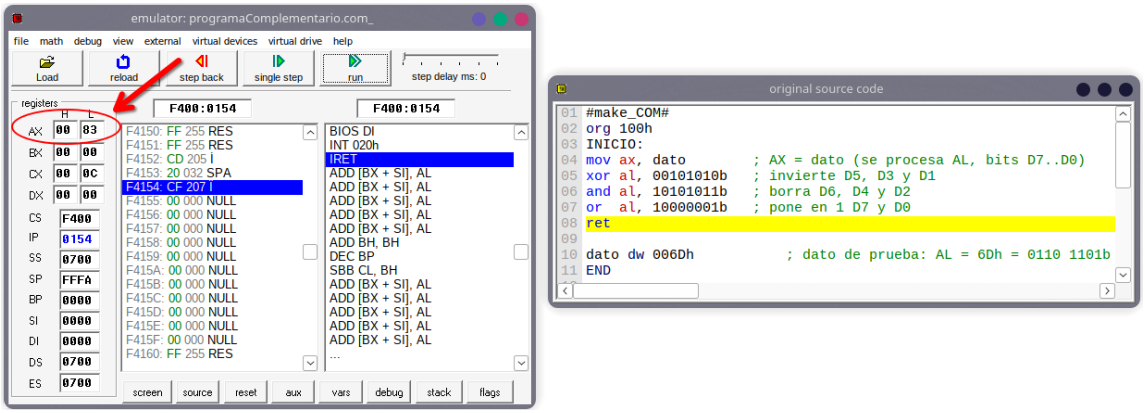


Figura 20: Registros del emulador al finalizar el programa de manipulación de bits.

El resultado final cumple el enunciado: D7 y D0 valen 1, D6, D4 y D2 valen 0, y D5, D3 y D1 quedaron invertidos respecto al dato original.

6. Conclusiones

- Las instrucciones lógicas operan bit a bit, y combinadas con una máscara permiten borrar (`AND`), poner en 1 (`OR`) o invertir (`XOR`) bits específicos de un dato sin afectar a los demás.
- El registro de banderas refleja el resultado de la última operación que lo modifica: Z indica resultado cero, S copia el bit más significativo del tamaño de la operación y P indica un número par de unos en el byte bajo. `AND`, `OR` y `XOR` dejan siempre C y O en 0.
- `XOR` de un registro consigo mismo lo pone a cero y activa Z, mientras que `AND` y `OR` de un registro consigo mismo no cambian su valor.
- `XOR` con FFh y `NOT` producen el mismo resultado, pero solo `XOR` actualiza las banderas; `NOT` no modifica ninguna.
- Algunas banderas (C, D e I) se pueden activar o desactivar directamente con instrucciones específicas (`STC/CLC`, `STD/CLD`, `STI/CLI`), independientemente del resultado de una operación.
- `TEST` permite probar bits con una máscara sin destruir el dato, a diferencia de `AND`. La máscara debe tener 1 en los bits que se desean probar.
- Las directivas `DB` y `DW` permiten declarar datos de 8 y 16 bits que quedan almacenados en memoria junto al programa, y que el procesador guarda en formato *little endian*.